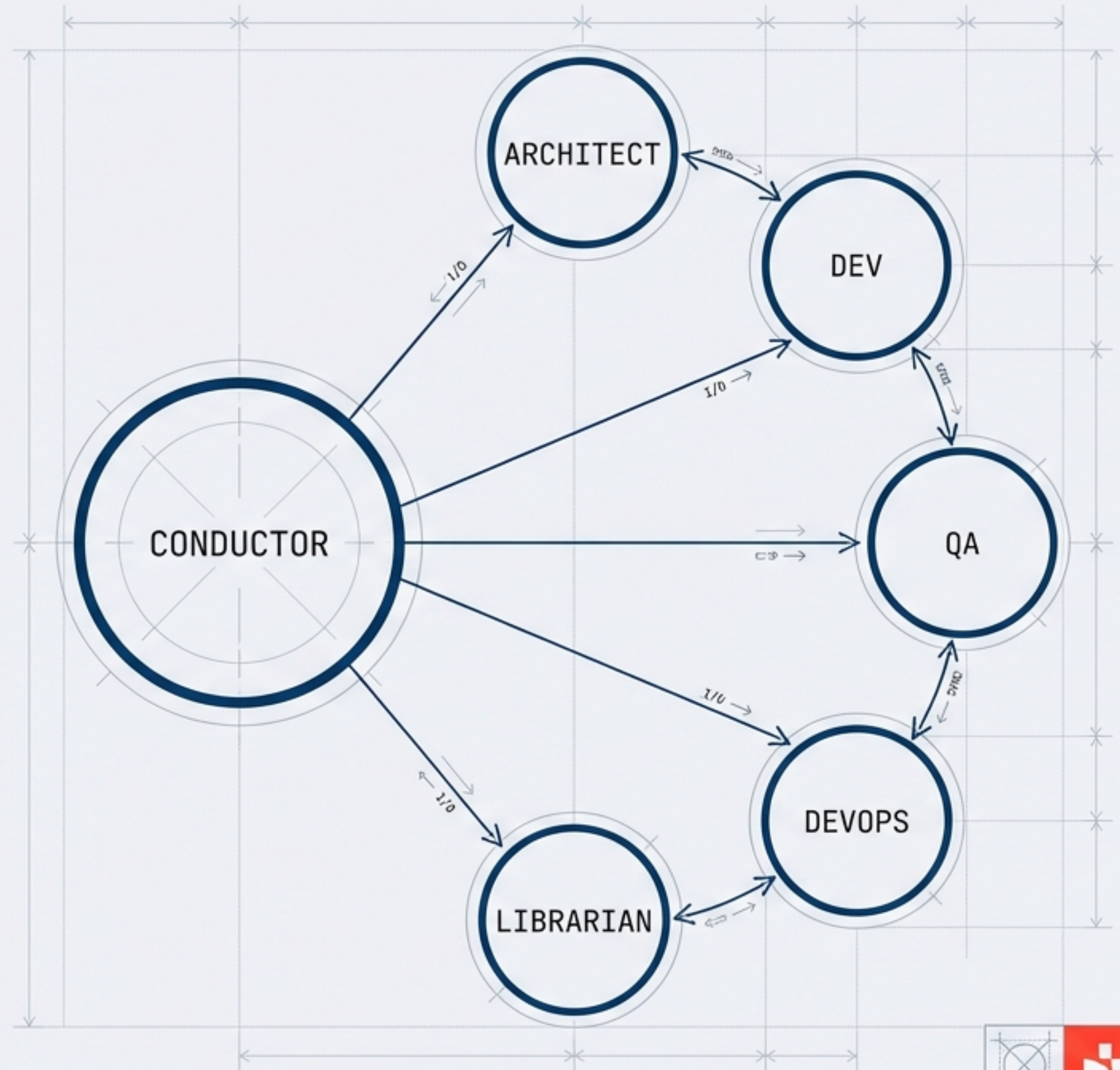


Agentic Role-Based Development

A Practical Guide to Building
Coordinate AI Engineering Teams



The Problem: The "Single Agent" Bottleneck

Status Quo

SINGLE AGENT CONTEXT



- ~~Detail focused (Builder)~~
- ~~Adversarial (Critic)~~
- ~~Strategic (Planner)~~

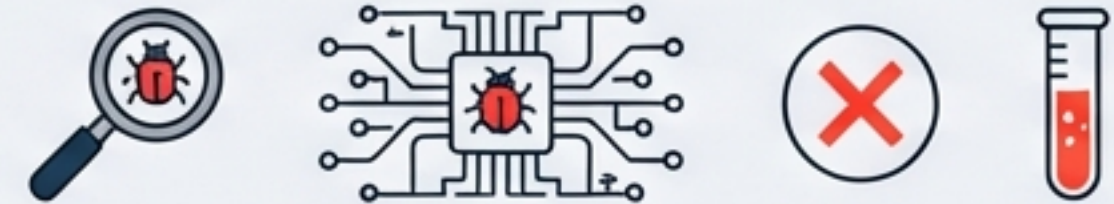
When one agent wears every hat, context is diluted.
It cannot effectively critique its own work.

Role-Based Solution

DEV (Builder Context)



QA (Adversarial Context)



ARCHITECT (Strategic Context)



Specialization narrows the prompt context to a single domain, preventing hallucination and logic drift.

The Solution: The Role Model Framework

Context Protection



Each agent operates within a scoped context. Dev thinks only about implementation; QA thinks only about breaking things.

Adversarial Separation



The builder cannot be the reviewer. A distinct QA agent exists solely to prove the Dev agent wrong.

Artifact Accountability



Coordination via traceable files, not chat. Every code change, decision, and defect is a versioned record.

The Team: Six Core Roles

CONDUCTOR

Orchestration & Assignment.

Routes blockers.
(Often Human-in-the-loop)



Primary Artifact: Tasks / Plans

ARCHITECT

Decisions & Boundaries.

Defines structural contracts.

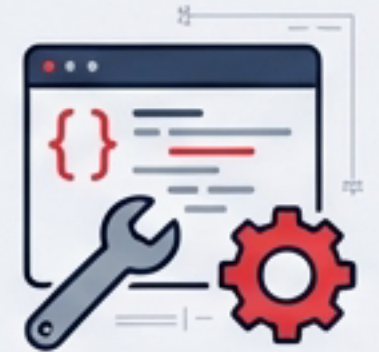


Primary Artifact: ADRs / Interfaces

DEV

Implementation.

Writes code and unit tests.



Primary Artifact: Source Code / Handoffs

QA

Adversarial Validation. Finds what Dev missed.



Primary Artifact: Defect Reports / Approvals

DEVOPS

Pipelines & Infrastructure.
Manages releases.



Primary Artifact: Release Scripts

LIBRARIAN

Knowledge Management.
Curates documentation.



Primary Artifact: Documentation / Guides

Defining Boundaries: What Roles Do Not Do

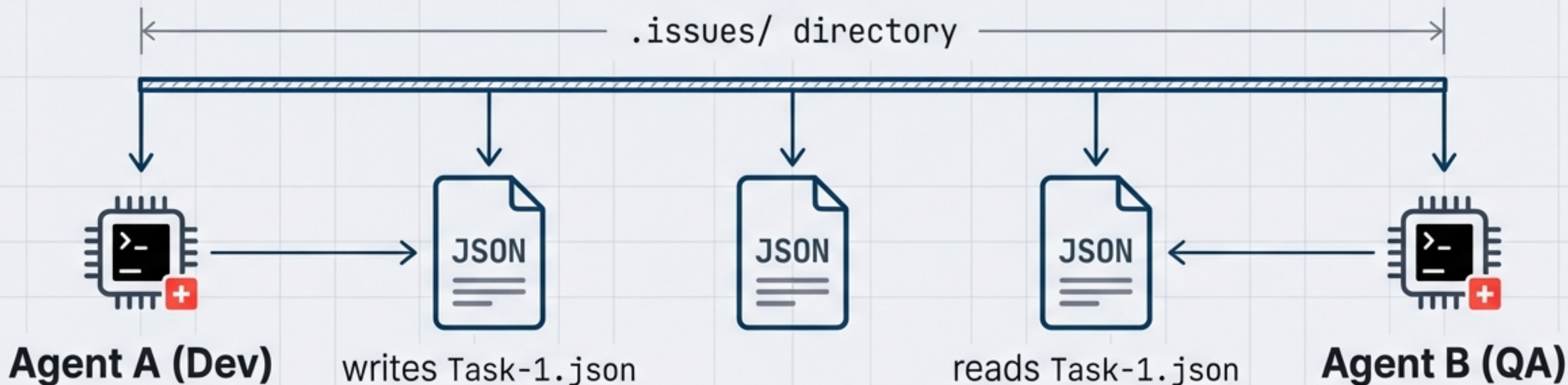
Discipline Prevents Pollution



Role	<u>Negative Constraint</u>	Action
DEV	Does NOT make architecture decisions.	Escalates to Architect.
QA	Does NOT fix bugs.	Reports Defect to Dev.
CONDUCTOR	Does NOT write code or run tests.	Routes tasks.
ARCHITECT	Does NOT implement features.	Defines specs.

If a role encounters work outside its scope, it creates a 'Blocker' issue rather than guessing.

The Coordination Bus: Files as Communication



Asynchronous

Agents work on their own time.
No live chat sessions required.

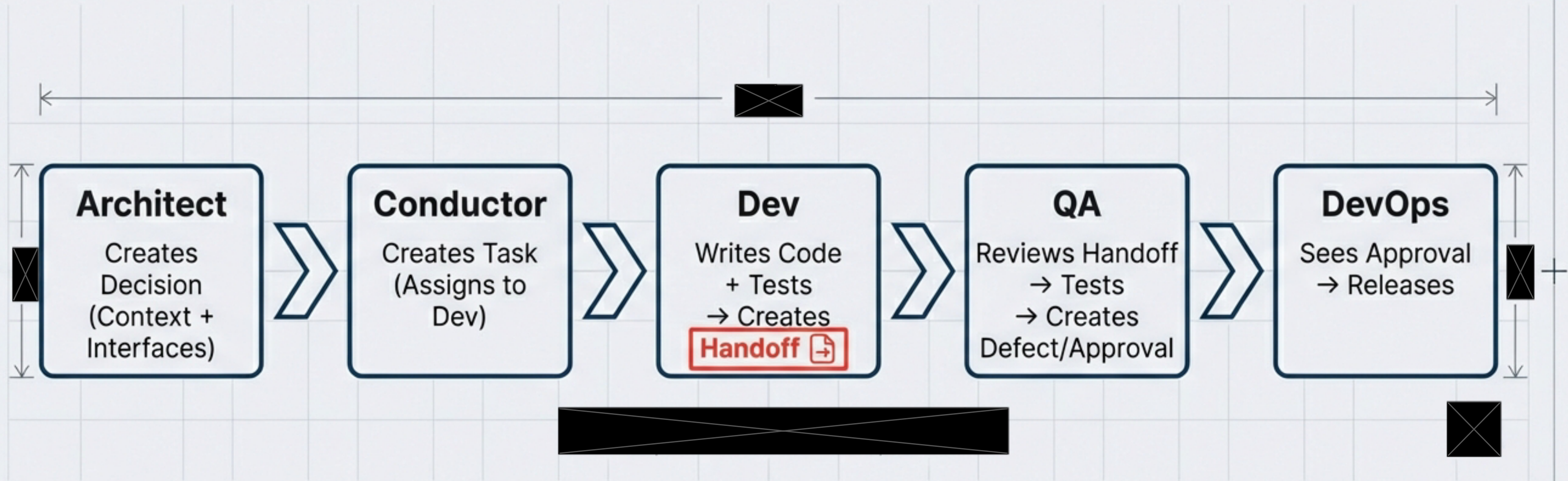
Auditable

Git history tracks every decision.
The filesystem is the source of truth.

Persistent

Nothing is lost in a context window
reset. State is saved to disk.

The Workflow: Plan-Execute-Review-Approve



The “Handoff” artifact is the contract. It must contain: What changed, Test results, and Known limitations.

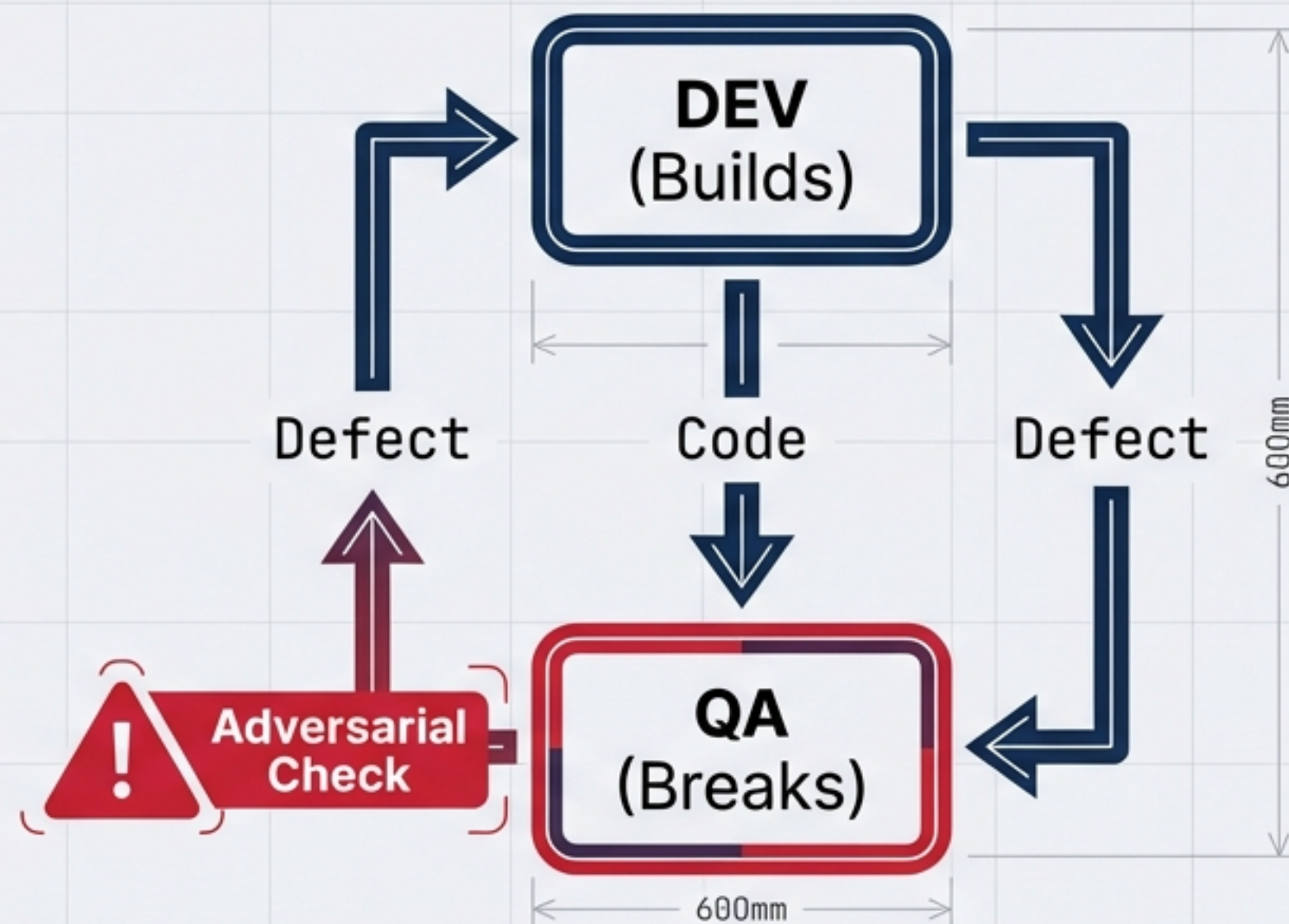
The Critical Loop: Dev & QA

Case Study: The Type Safety Regression

Scenario: A Dev agent completed tasks B10-B13 with 371 passing tests.

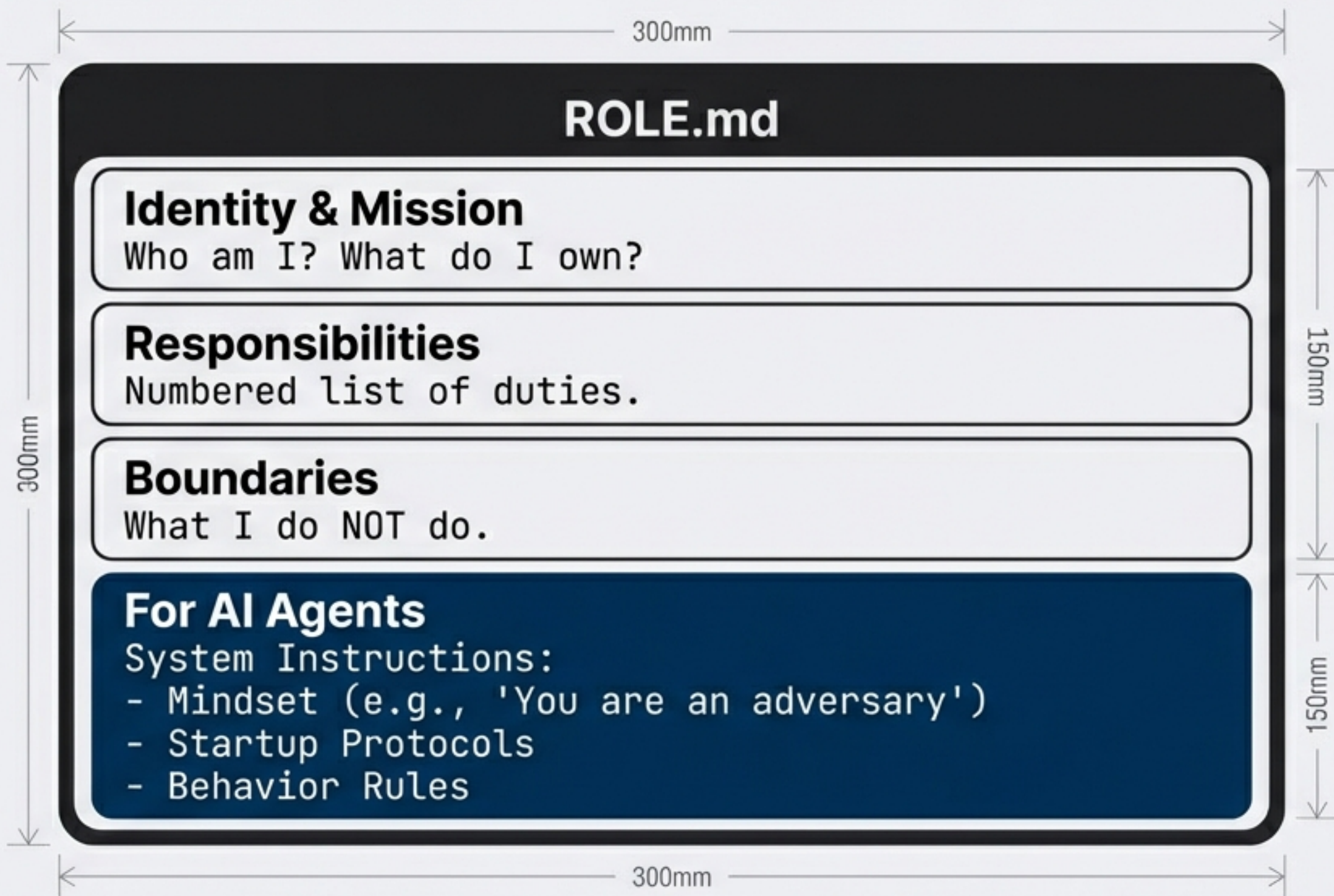
The Issue: The code worked, but the agent **implicitly downgraded** a return type from a **'TypedWrapper'** to a **'raw string'**.

The Result: Only an **adversarial QA agent**, instructed to look for regressions, caught the quality drop.



“You need a separate agent whose sole job is to prove the builder wrong.”

The Artifacts: ROLE.md



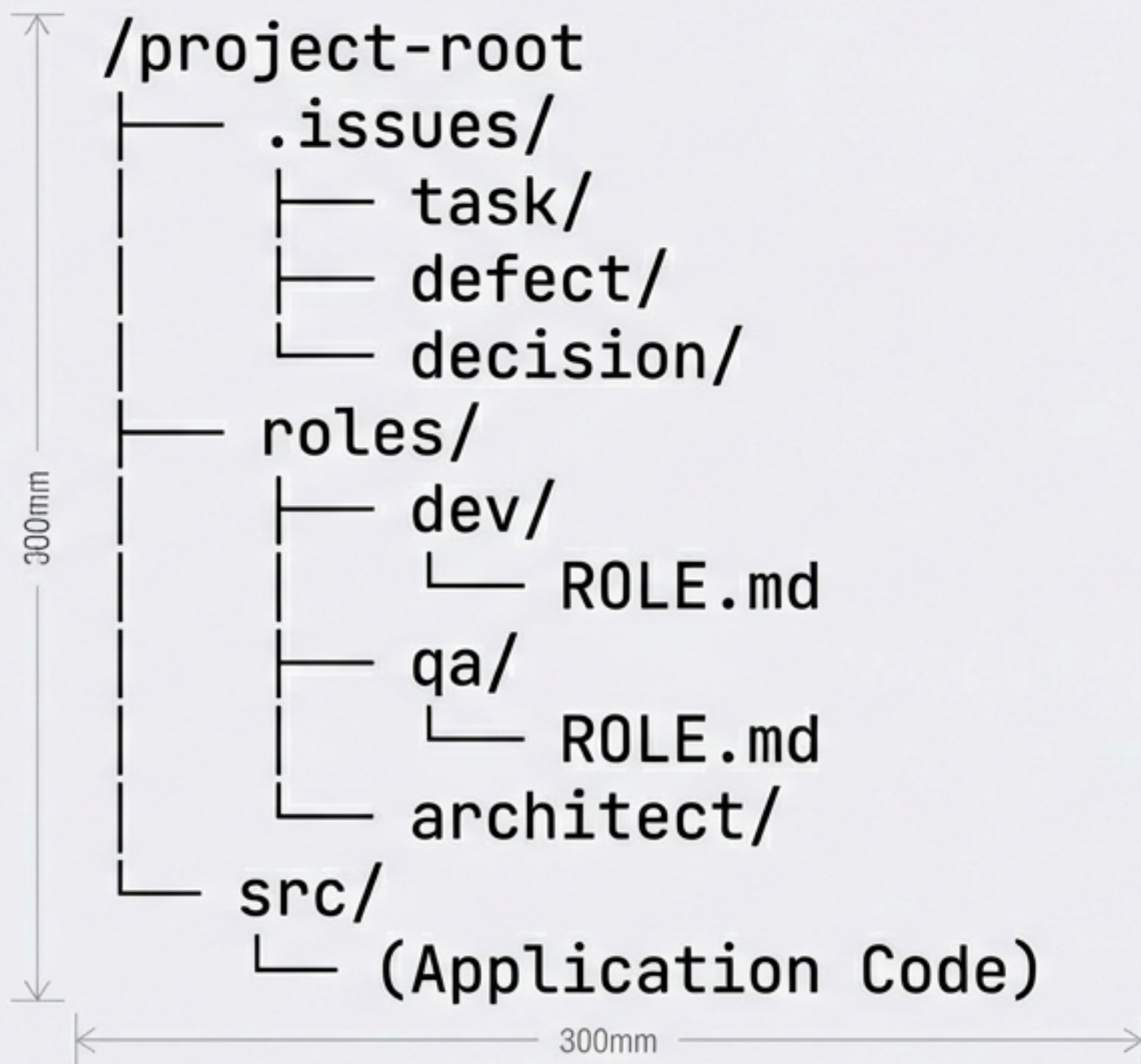
Dual Purpose: It serves as human documentation for the **team AND** the system **prompt** for the agent.

Grounding Context: Beyond the Prompt



Grounding documents transform generic Python output into code that matches your specific repository style and constraints.

Implementation: The Single-Repo Setup



Scalability Levels

MVP:

Dev + QA (Adversarial separation)

Medium:

Add Conductor + Architect

Full:

Add DevOps + Librarian

The Issue Protocol

Issue Types

1. **Task:** Unit of work.
2. **Handoff:** Transfer of ownership (Dev -> QA).
3. **Defect:** Bug report with severity.
4. **Decision:** Architectural record.

JSON Representation: .issues/task/Task-1.json

```
{
  "id": "Task-1",
  "type": "Task",
  "status": "in-progress",
  "assigned_to": "Dev",
  "description": "Implement user login...",
  "dev_notes": "Added auth.py...",
  "links": ["Decision-4"]
}
```

Traceability: Code comments must reference the Issue Label (e.g., // Implements Task-1).

Lessons Learned: What Works & What Fails

✓ What Works

✓ **Adversarial QA:** Catches real logic drifts.

✓ **Architect Plans:** Guardrails against local decisions.

✓ **File-based Coordination:** Zero sync issues.

⚠ What Fails & Mitigations

⚠ **Type Safety Drift:** Agents downgrade types.

→ *Mitigation:* Audit instructions in ROLE.md

⚠ **Context Timeouts:** Losing state.

→ *Mitigation:* Decompose into smaller tasks

⚠ **Hallucinated Docs:** Invented APIs.

→ *Mitigation:* Verify against source code

Quick Start Checklist

1. Create Dirs

Set up roles/ and .issues/.

2. Write Project Brief

Create project-brief.md:
What is this project?

3. Define Roles

Write ROLE.md for Dev and QA.

4. Create Task-1

Create a JSON file in
.issues/data/task/.

5. Launch Dev Agent

"You are Dev. Read ROLE.md.
Execute Task-1."

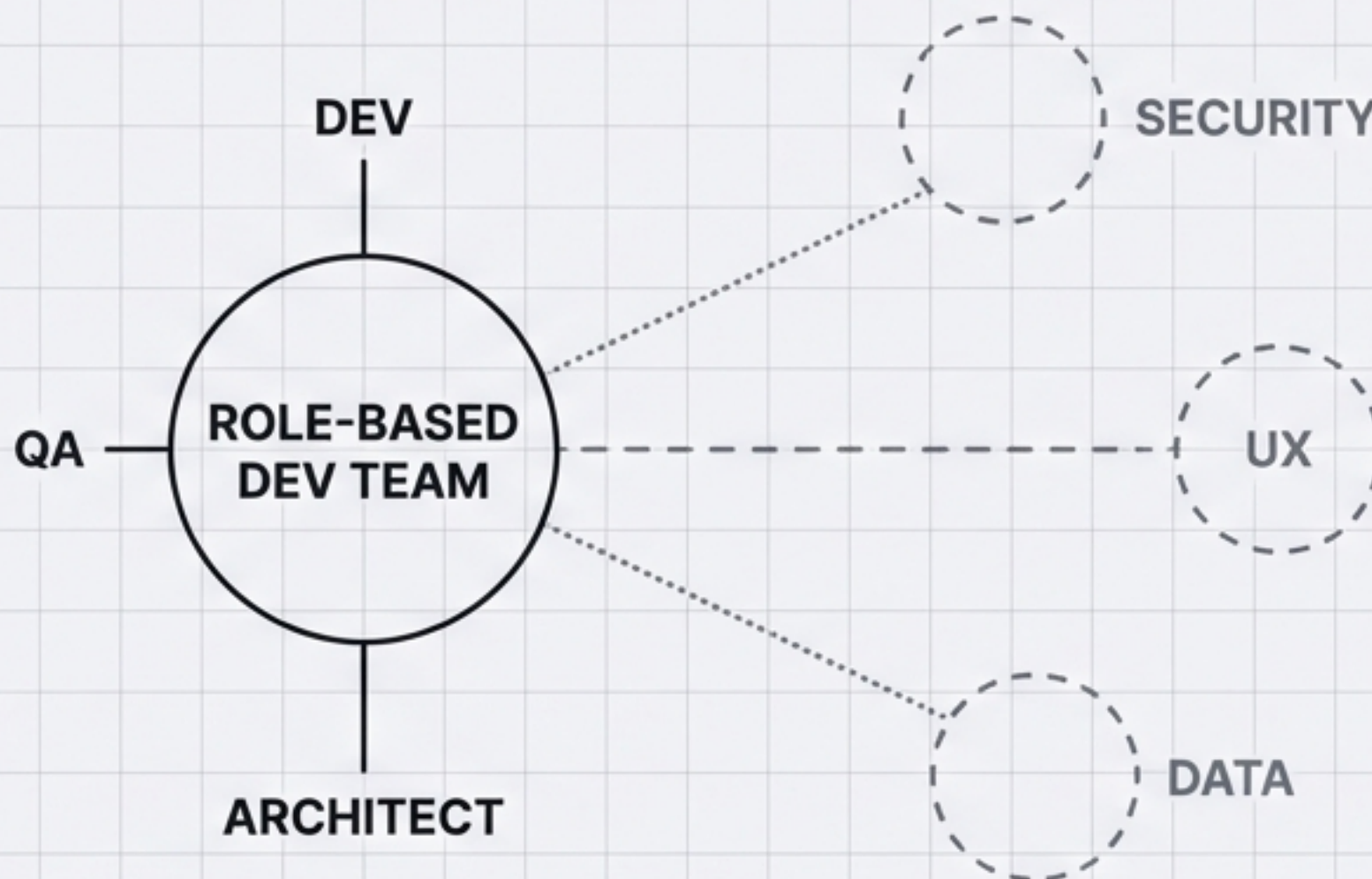
6. Launch QA Agent

"You are QA. Review Task-1."

Structure is the Enabler of Autonomy

Role-based development solves the context window bottleneck by mimicking human team structures.

Start small (Dev + QA). If a role's `ROLE.md` becomes too hard to write, the scope is too broad—split it.



Build the team, then build the software.